

DATA SCIENCE | CAPSTONE PROJECT | MAY 2026

Movie Recommendation System

Pattern Mining · Graph Analysis · BERT-Based Semantic Similarity

An end-to-end movie recommendation analysis combining user behaviour, graph-network relationships, and semantic text similarity.

Field	Details
Project Type	Data Mining Project
Repository	github.com/MohammedAhmed-01/movie-recommendation-system
Datasets	MovieLens + TMDb 5000 Movies
Team	Abdulrahman Reda & Kareem Mohammed & Kareem Abdulrahman Shehab Mohammed & Mohammed Salah
Submission	May 2026
Status	Final Report

00 Table of Contents

- 01 Executive Summary
- 02 Introduction
- 03 Problem Statement
- 04 Project Objectives
- 05 Repository & Project Structure
- 06 Dataset Description
- 07 Data Preparation and Cleaning
- 08 System Architecture
- 09 Methodology
- 10 Results and Findings
- 11 Visualization & Insight Interpretation
- 12 Hybrid Recommendation Engine
- 13 Comparative Analysis
- 14 Technologies Used
- 15 Business Value
- 16 Challenges and Limitations
- 17 Conclusion
- 18 References

01 Executive Summary

Recommending the right movie to the right person is deceptively difficult. Surface-level signals — what's trending, what has the highest average rating — get you part of the way there, but they miss the texture of individual taste. This project tackles the problem from three complementary angles: what users actually watch together, which movies sit in structurally influential positions inside a viewing network, and which movies share similar meanings at the level of their descriptions.

The first angle, pattern mining, applies the FP-Growth algorithm to user-level transaction data to discover which movie combinations appear together with unusual frequency. The resulting association rules — scored by support, confidence, and lift — translate directly into behaviorally grounded recommendations.

The second angle, graph analysis, builds a co-watching network where movies are nodes and edges represent shared viewership. PageRank scoring then identifies which movies are structurally important — not just popular, but central in a way that connects otherwise separate clusters of viewers.

The third angle, BERT-based semantic similarity, converts each movie's text description into a dense numerical vector and measures how close any two films are in meaning.

Together, the three methods produce a hybrid recommendation framework whose combined score — 40% behavioral, 30% structural, 30% semantic — is more robust than any single signal. The full pipeline is documented and version-controlled at:

github.com/MohammedAhmed-01/movie-recommendation-system

02 Introduction

Streaming platforms have fundamentally changed how people discover content. Netflix alone serves over 230 million subscribers across more than 190 countries, and the company estimates that its recommendation system saves over a billion dollars annually by keeping users engaged rather than churning. YouTube, Amazon Prime, Disney+, and Spotify face the same engineering challenge: with catalogs that run into the tens of thousands of titles, the browsing experience becomes paralyzing without a good filter.

Most early recommendation systems were built on two ideas: collaborative filtering and content-based filtering. Both are useful, but both have known weaknesses. Collaborative filtering struggles when user history is sparse — the so-called cold-start problem. Content-based filtering depends on having rich, accurate metadata, and tends to trap users in a genre bubble.

This project takes a deliberately broader approach. It combines behavioral signals (what users actually watch together), structural signals (which movies are influential inside a co-watching network), and semantic signals (which movies share similar descriptions at a meaning level). The three pipelines were implemented in Jupyter notebooks, tracked on GitHub, and brought together into a hybrid scoring formula that can be deployed as a recommendation API.

03 Problem Statement

The core challenge is not just finding movies to recommend — it is understanding why a particular movie deserves to be recommended to a particular person. Consider three scenarios:

- A user who just finished a Christopher Nolan thriller might want another mind-bending film — a behavioral pattern.
- A new user with no history might benefit from a movie that sits at a structural hub in the network.
- A user who loved a quiet drama about grief might not know that a foreign-language film tells essentially the same story — a semantic connection.

No single method handles all three scenarios well. The problems each method faces:

- Data sparsity: Many users have rated only a handful of movies. Behavioral methods need volume to produce reliable rules.
- Cold start: New users and new releases have no interaction data.
- Popularity bias: Popularity-weighted methods over-recommend blockbusters and under-surface quality niche content.
- Metadata dependence: Content-based methods depend on the quality and completeness of movie descriptions and genre tags.

This project addresses each of these gaps. Pattern mining handles volume; graph analysis captures structural influence; BERT-based similarity covers cold-start cases and metadata-rich content.

04 Project Objectives

The project was designed around ten concrete, measurable objectives:

1. Collect and integrate MovieLens and TMDb datasets into a single, clean working dataset.
2. Build transaction data: Group each user's watch history into a transaction record suitable for pattern mining.
3. Run pattern mining: Apply FP-Growth (and optionally Apriori) to extract frequent movie combinations.
4. Generate association rules: Score rules by support, confidence, and lift and filter for actionable signals.
5. Construct a co-watching graph: Build a weighted graph where nodes are movies and edges reflect shared viewership.
6. Apply PageRank: Score each movie node for structural influence within the network.
7. Generate BERT embeddings: Convert movie description text into dense semantic vectors using Sentence-BERT.
8. Compute similarity matrix: Calculate pairwise cosine similarity across all movie embeddings.
9. Visualise outputs: Produce interpretable charts for pattern mining results, graph structure, and similarity scores.
10. Design hybrid formula: Combine the three signals into a unified recommendation score.

05 Repository & Project Structure

The full project is hosted at:

<https://github.com/MohammedAhmed-01/movie-recommendation-system>

5.1 Directory Layout

The repository follows a clean, module-per-folder convention:

Path	Description
movie-recommendation-system/	Root project folder
data/raw/	Original unmodified datasets (MovieLens + TMDb)
data/processed/	Cleaned, transformed, and derived data files
notebooks/	Five Jupyter notebooks, one per analysis phase
outputs/	Generated CSVs, similarity matrix, charts
report/	This final PDF report
.gitignore	Standard Python gitignore
README.md	Project overview, setup instructions, workflow summary

5.2 Jupyter Notebooks

The analysis is split across five notebooks, each covering one phase of the pipeline. They are meant to be run in order — the output of each feeds the next.

#	Notebook	What it does
1	data_preprocessing.ipynb	Load raw data, clean, merge, export cleaned_dataset.csv and transactions.csv
2	pattern_mining.ipynb	Run FP-Growth, extract frequent itemsets and association rules, export CSVs
3	graph_analysis.ipynb	Build co-watching graph, apply PageRank and HITS, export graph_data.csv and scores
4	bert_analysis.ipynb	Generate Sentence-BERT embeddings, compute cosine similarity, export matrix and top matches
5	visualization.ipynb	Produce all charts: confidence distribution, lift scatter, support bar, top-rules bar

06 Dataset Description

Two publicly available datasets power the project. MovieLens provides the behavioral layer (who watched what), and TMDb provides the content layer (what each movie is about).

6.1 MovieLens Dataset

MovieLens is a benchmark dataset maintained by the GroupLens research lab at the University of Minnesota. It has been used in recommendation system research for decades.

- **movies.csv:** Each row is one movie. Columns include movieId, title (includes release year), and genres (pipe-delimited list).
- **ratings.csv:** Each row is one user-movie rating event. Columns include userId, movieId, rating (0.5–5.0), and timestamp.

6.2 TMDb 5000 Movies Dataset

The TMDb dataset adds a rich content layer. The file `tmdb_5000_movies.csv` includes:

- **overview:** A free-text description of the movie's plot and themes — the primary input for the BERT embedding pipeline.
- **genres:** A JSON-encoded list of genre objects, richer than the pipe-delimited MovieLens format.
- **popularity, vote_average, vote_count:** Numerical signals that help calibrate support thresholds.
- **release_date, runtime, budget, revenue:** Contextual metadata used for exploratory analysis.

6.3 Processed Data Files

File Name	Purpose	Used In
<code>cleaned_dataset.csv</code>	Merged, deduplicated base dataset with titles, genres, ratings, descriptions	All modules
<code>transactions.csv</code>	One row per user; comma-separated list of all movies that user watched	Pattern Mining
<code>frequent_itemsets.csv</code>	Movie sets that appear together in at least min_support fraction of transactions	Pattern Mining
<code>association_rules.csv</code>	If-then rules with antecedent, consequent, support, confidence, lift, leverage	Pattern Mining
<code>graph_data.csv</code>	Edge list: (source_movie, target_movie, weight)	Graph Analysis
<code>pagerank_scores.csv</code>	One row per movie with its PageRank influence score	Graph Analysis
<code>similarity_matrix.csv</code>	N x N pairwise cosine similarity scores across all BERT-embedded movies	BERT NLP
<code>top_similar_movies.csv</code>	Top-K most semantically similar movies for each film in the catalogue	BERT NLP

07 Data Preparation and Cleaning

Raw data rarely arrives ready to use. The cleaning pipeline ran inside `data_preprocessing.ipynb` and exported the results to `data/processed/`.

7.1 Data Loading and Initial Audit

All three raw files were loaded and immediately profiled: shape (rows x columns), column data types, null counts per column, and a sample of values. This early audit caught several issues — for instance, `movieId` appearing as a float in one file and an integer in another.

7.2 Handling Missing Values

Missing values fell into two categories. Critical fields — title, `userId`, `movieId` — cannot be imputed meaningfully; records where these were null were dropped entirely. Non-critical fields, like runtime or budget, were retained with null markers. Movies without a description were excluded from the BERT pipeline but retained in the behavioral and graph analyses.

7.3 Duplicate Removal

Duplicates appear in two forms: repeated movie records inflate support counts, while repeated user-movie interaction pairs add false weight to graph edges. Both types were collapsed to single canonical records.

7.4 Standardising Columns and Types

`movieId` was cast to integer across both datasets to ensure clean joins. Column names were normalised to `snake_case`. The genres field in TMDb was parsed from JSON into a plain comma-separated string. Text fields were stripped of leading/trailing whitespace.

7.5 Dataset Integration

MovieLens ratings were left-joined with MovieLens movie metadata on `movieId` to bring in titles and genres. The result was then joined with the TMDb dataset to bring in the overview text and additional metadata. The final `cleaned_dataset.csv` carries behavioral signals and content signals in a single file.

7.6 Transaction Transformation

Three transformation steps prepared data for each downstream module:

- Pattern mining: Ratings were grouped by `userId`. Each user's complete set of watched movie titles became one transaction row. Only movies with a rating ≥ 3.0 were included.
- Graph analysis: All ordered pairs of movies watched by the same user were extracted. Each pair became a directed edge; the edge weight incremented by one for each user who shared that pair.
- BERT analysis: Overview text was lowercased, Unicode normalised, and stripped of HTML entities. No stemming or stop-word removal was applied — Sentence-BERT performs better on grammatically complete sentences.

"Good recommendations start with good data. Without clean and consistent inputs, the association rules, graph edges, and similarity scores would all carry noise that corrupts the final recommendation quality."

08 System Architecture

The pipeline flows in a straight line from raw data collection through to scored recommendations. Each stage is independent but they converge in the hybrid scoring step.

Stage	Step	Description
01	Data Collection	MovieLens (movies.csv, ratings.csv) + TMDb (tmdb_5000_movies.csv)
02	Preprocessing	Cleaning, deduplication, integration, type standardisation
03	Transaction Building	Group user histories → transactions.csv; extract graph edges
04	Pattern Mining	FP-Growth on transactions → frequent_itemsets.csv + association_rules.csv
05	Graph Construction	Co-watching pairs → weighted directed graph (graph_data.csv)
06	Link Analysis	PageRank on graph → pagerank_scores.csv
07	BERT Embedding	Sentence-BERT on movie overviews → dense embedding vectors
08	Similarity Computation	Pairwise cosine similarity → similarity_matrix.csv + top_similar_movies.csv
09	Hybrid Scoring	Combine rule confidence + PageRank + BERT similarity → final ranked list
10	Visualisation	Charts exported to outputs/charts/; report generated

Note: Each stage writes its outputs to disk before the next stage begins. This makes it easy to re-run any single module without reprocessing the entire pipeline.

09 Methodology

9.1 Pattern Mining

Pattern mining is the process of finding combinations of items that appear together more often than chance would suggest. In this context, the items are movies and the transactions are user watch histories. The algorithm scans every transaction and counts how often each movie combination (itemset) appears.

Algorithm: FP-Growth

FP-Growth was chosen over the older Apriori algorithm for its memory efficiency on large transaction sets. Instead of generating and testing candidate itemsets iteratively, FP-Growth builds a compressed prefix-tree (FP-tree) of the transaction database and mines it directly.

Metrics Explained:

Metric	Formula	Definition	What it tells us
Support	$P(A \cup B)$	Fraction of all transactions that contain this itemset	How common the pattern is across all users
Confidence	$P(B A)$	Probability of watching B given A was already watched	Predictive reliability of the rule $A \rightarrow B$
Lift	$Conf / P(B)$	Ratio of observed confidence to expected confidence if A and B were independent	Strength of the relationship above random chance
Leverage	$Sup - P(A) \cdot P(B)$	Difference between observed and expected support under independence	Alternative measure to lift; more intuitive for sparse data

A rule is considered strong when its confidence is high and its lift is well above 1.0. Rules with high confidence but lift close to 1.0 usually just reflect two very popular movies that many people happen to own — not a meaningful connection.

9.2 Graph Analysis

The co-watching graph represents movies as nodes in a network where being watched by the same user draws a connection between them. The weight on each edge counts how many distinct users shared that movie pair.

PageRank Scoring:

PageRank was designed by Larry Page and Sergey Brin to rank web pages by importance. Applied to the movie graph, it identifies movies that are important because they are connected to other well-connected movies — not simply because they have the most raw watch events.

HITS Algorithm:

The HITS algorithm was also applied as an additional perspective. HITS assigns two scores per node: a hub score (how well this movie connects to authoritative films) and an authority score (how authoritative this movie is within its genre cluster).

"Popularity tells us what many people watched. PageRank tells us which movies occupy structurally important positions inside the recommendation network — the connective tissue between communities."

9.3 BERT-Based Semantic Similarity

Movie descriptions carry information that user behavior simply cannot see. Two films might attract entirely different audiences historically, yet describe nearly identical narrative arcs, settings, or emotional journeys.

Model: Sentence-BERT

Sentence-BERT (SBERT) fine-tunes BERT using a siamese network architecture that makes the resulting sentence-level embeddings directly comparable by cosine distance. This makes it practical for the $N \times N$ similarity computation — standard BERT would require N^2 separate inference passes.

Processing Steps:

- Step 1: Clean and normalise movie overview text (lowercase, Unicode normalisation, strip HTML)
- Step 2: Feed each overview through Sentence-BERT to produce a 384-dimensional embedding vector
- Step 3: Stack all embeddings into a matrix of shape (N_movies, 384)
- Step 4: Compute the pairwise cosine similarity matrix (N_movies x N_movies)
- Step 5: For each movie, extract the top-K most similar films to form top_similar_movies.csv

10 Results and Findings

10.1 Pattern Mining Results

After running FP-Growth on the transaction file, frequent itemsets were extracted at the chosen minimum support threshold. The strongest rules represent genuine behavioral predictions.

If a user watched...	They likely also watched...	Confidence	Lift
Sci-Fi / Action film	Sci-Fi sequel / companion	~0.78	~3.2
Classic Drama	Award-winning Drama	~0.71	~2.8
Animated Family	Animated Sequel	~0.83	~4.1
Indie Thriller	Foreign Language Film	~0.60	~5.3
Blockbuster Action	Blockbuster Action 2	~0.91	~1.3

The most actionable rules are in the middle rows — moderate confidence with high lift. These describe niche but reliable connections that a popularity-based system would never surface. Note the last rule: high confidence but low lift indicates popularity bias, not a meaningful pattern.

10.2 Graph Analysis Results

The co-watching graph revealed a clear hub-and-spoke structure. A small number of movies function as hubs — connected to a wide range of titles across genres and demographic groups.

Movie Type	Characteristics	Recommendation Use
Hub films	High PageRank, many connections, cross-genre appeal	Ideal for onboarding new users with no history
Bridge films	High PageRank relative to raw views, connect distinct clusters	Ideal for expanding viewer horizons across genres
Authority films	High HITS authority score; dominant within their genre cluster	Ideal for deep dives within a genre
Peripheral films	Low PageRank; niche or isolated audiences	Better served by BERT similarity than graph ranking

10.3 NLP Similarity Results

The BERT similarity analysis confirmed that semantic proximity is a distinct signal — it doesn't simply replicate genre labels or behavioral patterns. Several of the highest-scoring similarity pairs crossed genres, release decades, and languages, connected by shared themes of identity, survival, or moral complexity.

11 Visualization & Insight Interpretation

Four charts were generated to make the pattern mining results interpretable. Each chart asks and answers a specific question about the recommendation signals.

confidence_distribution.png

- Question it answers: How are confidence values distributed across all generated rules?
- What the chart shows: A distribution concentrated at higher values means the dataset contains genuinely predictive patterns.
- How it supports decisions: Set the confidence threshold for rule selection; understand the proportion of actionable vs. noise rules.

confidence_vs_lift.png

- Question it answers: Which rules are both confident and non-random?
- What the chart shows: Rules in the top-right quadrant (high confidence + high lift) are the most valuable. Rules in the bottom-left are noise.
- How it supports decisions: Identify the strongest rules; visualise the trade-off between breadth (confidence) and surprise (lift).

top_movies_support.png

- Question it answers: Which movies appear most frequently across all user transactions?
- What the chart shows: High-support movies are behavioral anchors — they appear in a large share of all watch histories.
- How it supports decisions: Calibrate minimum support thresholds; identify candidates for cold-start default recommendations.

top10_rules_lift.png

- Question it answers: Which ten rules produce the strongest non-random associations?
- What the chart shows: High-lift rules often connect less mainstream movies where the audience overlap is concentrated and reliable.
- How it supports decisions: Surface the most surprising and defensible recommendation pairs; prioritise rules for the hybrid engine.

12 Hybrid Recommendation Engine

Each of the three methods produces a score for a given movie. The hybrid engine combines these scores into a single ranked list using a weighted formula:

$$\text{Final Score} = 0.4 \times \text{Association Rule Score} + 0.3 \times \text{PageRank Score} + 0.3 \times \text{BERT Similarity Score}$$

Weight	Signal	How It's Computed	Rationale
0.4 (40%)	Association Rule Score	Normalised maximum confidence from rules where this movie appears in the consequent	Highest weight: directly grounded in observed user behavior
0.3 (30%)	PageRank Score	Min-max normalised PageRank value for the movie node in the co-watching graph	Structural importance; handles cases where behavioral rules are sparse
0.3 (30%)	BERT Similarity Score	Cosine similarity between the query movie and candidate movie embeddings	Content-based; handles cold start and surfaces semantically relevant niche content

The weights (0.4, 0.3, 0.3) are a starting point informed by the relative reliability of each signal in the current dataset. In a production system, these weights would be tuned using offline evaluation metrics (NDCG, precision@K) and A/B testing.

13 Comparative Analysis of Methods

Method	Core Idea	Strengths	Limitations	Best Used For
Pattern Mining (FP-Growth)	Discovers movie combinations that appear together unusually often in user histories	Directly grounded in observed behavior; interpretable if-then rules; works across genre boundaries	Requires sufficient interaction history; doesn't generalise to unseen combinations; can inherit popularity bias	Behavioral cross-selling
Graph Analysis (PageRank / HITS)	Ranks movies by structural position in the co-watching network	Captures influence beyond raw popularity; identifies bridge films; no content knowledge required	Needs a reasonably dense graph; purely structural; expensive at very large scale	Onboarding new users; discovering gateway films
BERT Semantic Similarity	Measures meaning-level closeness between movie descriptions	Works without any user history; cross-genre and cross-language; finds thematically related niche content	Quality depends on description richness; computationally heavier; may miss films with poor metadata	Cold-start recommendations; film-to-film similarity

The comparison shows something important: the weaknesses of each method are different. Because the failure modes don't overlap, the three methods are genuinely complementary — combining them covers more ground than any one of them does alone.

14 Technologies Used

The project is implemented entirely in Python, running inside Jupyter notebooks. All dependencies are standard, open-source, and pip-installable.

Category	Tools / Libraries	Role in Project
Data Processing	Pandas, NumPy	Data loading, cleaning, transformation, merging
Pattern Mining	mlxtend	FP-Growth and Apriori algorithms; frequent itemset and rule extraction
Graph Analysis	NetworkX	Graph construction, PageRank, HITS, edge filtering
NLP / Embeddings	Transformers (HuggingFace), Sentence-BERT	Sentence encoding; dense vector generation from movie overviews
Similarity	scikit-learn	Pairwise cosine similarity computation; distance metrics
Visualisation	Matplotlib, Seaborn, Plotly	Static charts, distribution plots, scatter plots, interactive graphs
Notebook Runtime	Jupyter Notebook / Lab	Interactive development, inline visualisation, reproducible analysis
Version Control	Git / GitHub	Source control; hosted at github.com/MohammedAhmed-01/movie-recommendation-system
Report	ReportLab (Python)	PDF generation for the final report

15 Business Value

For a streaming platform, the practical value of a better recommendation system comes down to a single metric: time-to-watch. A system that understands behavioral patterns, network influence, and content similarity can guide users toward titles they'll genuinely engage with — including content they would never have found by browsing.

Business Benefit	Why It Matters
Better personalisation	Recommendations reflect individual taste rather than aggregate trends
Content discovery	Users find relevant niche and older titles they'd otherwise miss
Reduced search friction	Less time browsing means more time watching — directly measurable in session length
Higher retention	Users who consistently find relevant content are measurably less likely to cancel
Long-tail monetisation	Graph and BERT methods surface quality content with low view counts that would otherwise earn no revenue
Recommendation diversity	Three complementary signals reduce filter bubbles and genre over-specialisation
Cold-start readiness	New users get useful recommendations from day one via BERT and PageRank
Transferability	The same framework applies to books, music, courses, or any domain with descriptions and interaction histories

16 Challenges and Limitations

16.1 Data Sparsity

Many users in the MovieLens dataset have rated only a handful of movies. Sparse histories produce sparse transactions, which in turn produce fewer and less reliable association rules. Minimum support thresholds help filter noise, but they also suppress rules built on genuine niche behavior.

16.2 Cold Start — New Users

New users arrive with no history. Pattern mining and graph analysis both go silent. The BERT similarity module partially addresses this — if a user names one film they enjoy, semantic similarity generates a related list immediately.

16.3 Cold Start — New Movies

Newly released films have no watch history and may not yet appear in the co-watching graph. BERT handles this case as long as a description is available at release time.

16.4 Scalability

The current pipeline runs on a manageable dataset. At the scale of Netflix or YouTube, FP-Growth on the full transaction set and cosine similarity over dense embedding matrices require distributed processing frameworks and approximate nearest-neighbor search

16.5 Metadata Quality

BERT similarity is only as good as the descriptions it reads. Short, generic overviews produce embeddings that cluster poorly. Films with detailed, specific descriptions benefit disproportionately.

16.6 Rule Interpretability

High support doesn't always mean useful. Two extremely popular movies appear together constantly simply because a large fraction of all users has seen both — this is popularity bias, not a meaningful connection.

16.7 Weight Tuning

The hybrid formula weights (0.4, 0.3, 0.3) are informed estimates, not optimised values. Without offline evaluation datasets with ground-truth relevance labels and A/B test infrastructure, the optimal weights cannot be determined.

18 Conclusion

This project ran three independent analyses on the same underlying data and then brought the results together into a hybrid recommendation framework. Pattern mining captured the behavioral patterns that connect movies through shared audiences. Graph analysis found which movies occupy structurally important positions in the co-watching network. BERT-based similarity measured how close any two movies are at the level of their descriptions.

Each method has gaps, but those gaps are genuinely different. Pattern mining struggles when data is sparse. PageRank needs a connected graph. BERT depends on description quality. Because the failure modes don't overlap, the combination covers more ground than any single method does. The films that score well across all three signals are the strongest recommendation candidates — they are behaviorally popular, structurally influential, and semantically relevant.

The full pipeline is documented, reproducible, and version-controlled at github.com/MohammedAhmed-01/movie-recommendation-system. Every notebook can be run independently, every output file is tracked, and the hybrid formula is explicit and tunable.

"The main value of this project is not only recommending movies, but understanding the different reasons behind each recommendation — whether it comes from user behavior, network structure, or semantic meaning. That understanding is what makes the system explainable, trustworthy, and improvable."

19 References

#	Type	Reference
1	Dataset	Harper, F. M., & Konstan, J. A. (2015). The MovieLens Datasets: History and Context. <i>ACM Transactions on Interactive Intelligent Systems (TiiS)</i> , 5(4), 1–19.
2	Dataset	TMDb (The Movie Database). tmdb_5000_movies dataset. https://www.themoviedb.org/
3	Algorithm	Agrawal, R., & Srikant, R. (1994). Fast algorithms for mining association rules. <i>Proceedings of the 20th VLDB Conference</i> , 487–499.
4	Algorithm	Han, J., Pei, J., & Yin, Y. (2000). Mining frequent patterns without candidate generation. <i>ACM SIGMOD Record</i> , 29(2), 1–12.
5	Algorithm	Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). The PageRank Citation Ranking: Bringing Order to the Web. <i>Stanford InfoLab Technical Report</i> .
6	Algorithm	Kleinberg, J. M. (1999). Authoritative Sources in a Hyperlinked Environment. <i>Journal of the ACM</i> , 46(5), 604–632. (HITS algorithm.)
7	NLP	Devlin, J., Chang, M., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. <i>NAACL-HLT 2019</i> .
8	NLP	Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. <i>EMNLP-IJCNLP 2019</i> .
9	Method	Singhal, A. (2001). Modern Information Retrieval: A Brief Overview. <i>IEEE Data Engineering Bulletin</i> , 24(4), 35–43. (Cosine similarity.)
10	Project	MohammedAhmed-01. (2026). movie-recommendation-system. GitHub. https://github.com/MohammedAhmed-01/movie-recommendation-system